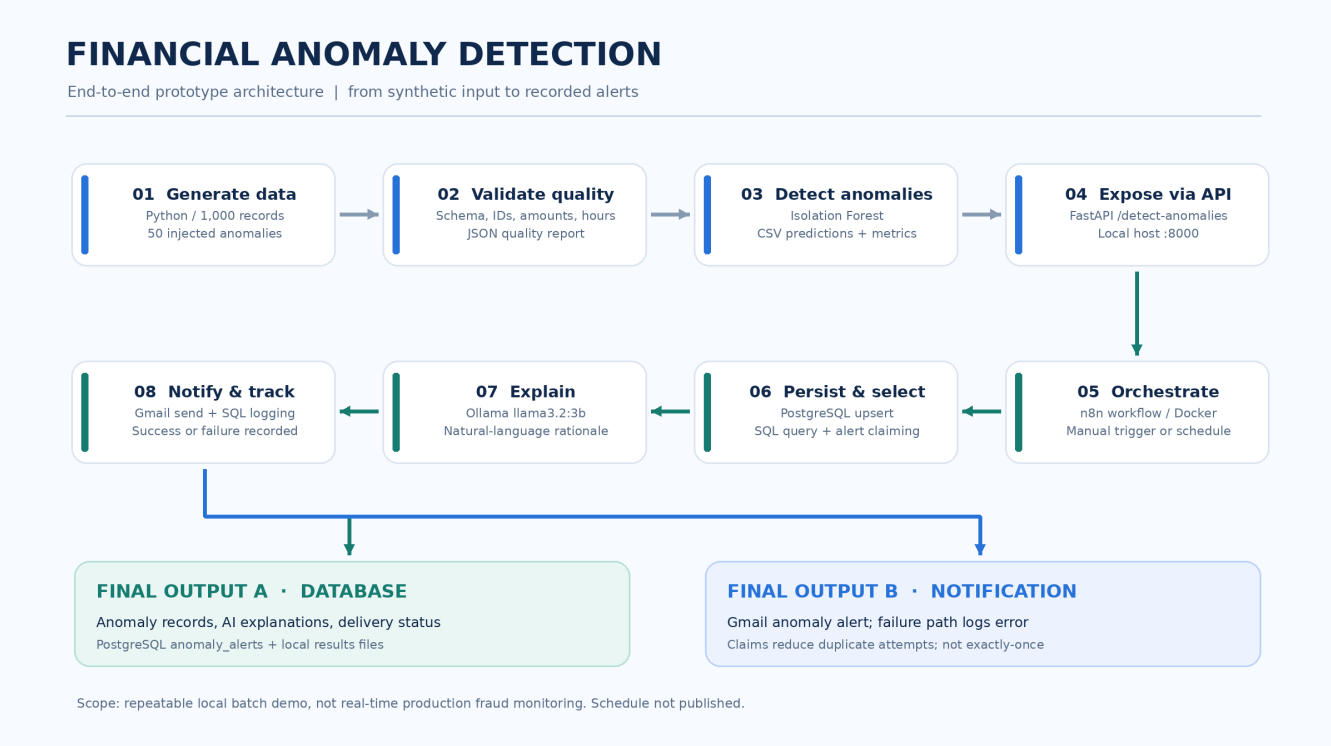
PROJECT TECHNICAL DOCUMENTATION

AI-Powered Financial Transaction Anomaly Detection Pipeline

Python | Isolation Forest | FastAPI | n8n | PostgreSQL | Ollama | Gmail



Portfolio prototype. A local batch-processing system that generates synthetic transactions, validates data quality, flags unusual records, creates AI explanations, stores alerts, and attempts email notification.

Prepared 31 October 2026 • Repository: github.com/durgapbattula-sys/ai-financial-anomaly-detection-n8n

1. Executive summary

The project demonstrates a complete anomaly-monitoring prototype across data engineering, machine learning, API integration, workflow orchestration, persistence, large language model explanations, and notification handling. Its purpose is to illustrate integration and observability, not to prove production fraud-detection performance.

Verified Python pipeline result	Value
Synthetic transactions	1,000
Injected anomaly labels	50
Detected anomalies	50
Data-quality issues	0
Precision / Recall / F1	0.980 / 0.980 / 0.980

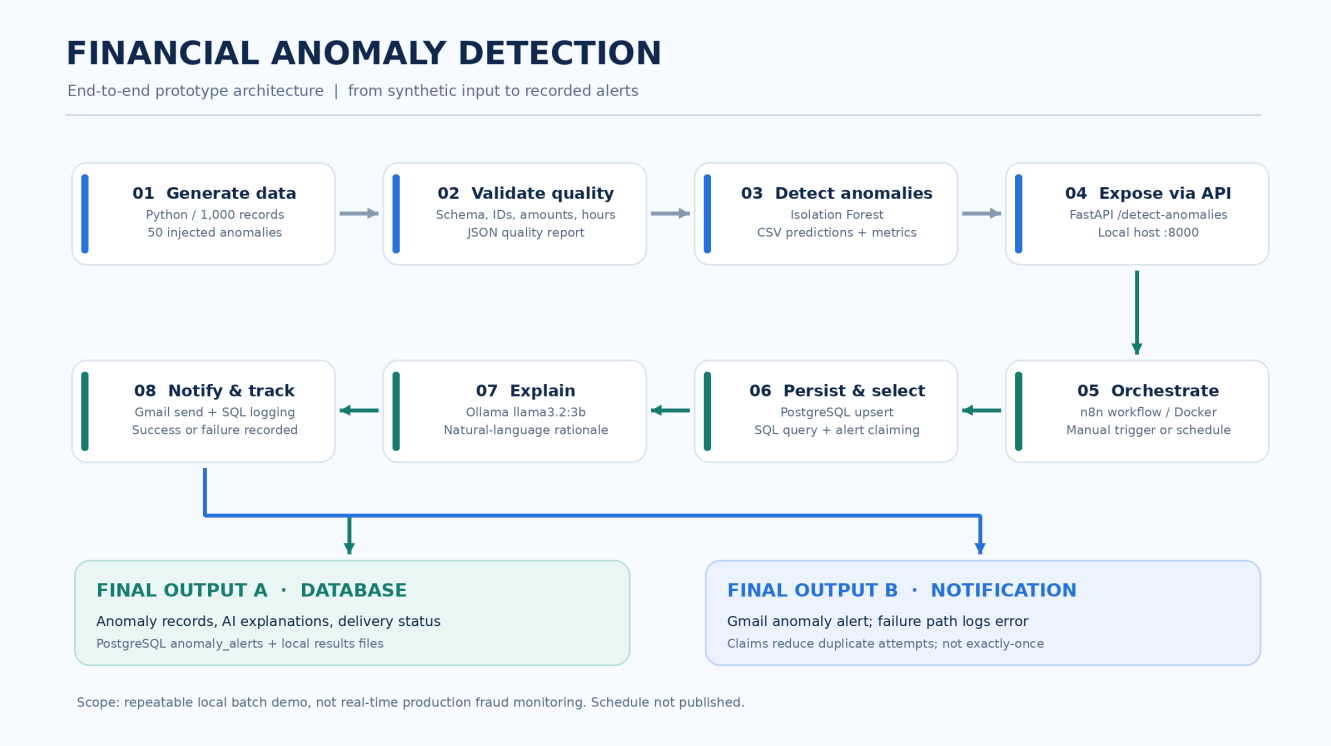
2. Problem and scope

Financial transaction data can contain unusual amounts or transaction hours that merit review. Manual inspection does not scale well; this prototype creates a repeatable batch and uses an unsupervised model to prioritize potentially anomalous rows. The pipeline does not identify confirmed fraud and does not use a live bank feed.

3. Technology stack

Component	Role
Python, pandas, NumPy	Synthetic dataset generation and transformation
Data-quality checks	Required columns; identifiers; amount and hour validation
scikit-learn Isolation Forest	Unsupervised anomaly scoring using amount and hour
FastAPI + Uvicorn	HTTP endpoint for n8n integration
n8n in Docker	Triggering, routing, branching, retry/error paths
PostgreSQL	Transaction/alert upserts, SQL selection, email claim state
Ollama llama3.2:3b	Human-readable explanations
Gmail	Outbound anomaly emails; success and failure routing

4. Architecture from first step to final output



The diagram is a conceptual architecture illustration, not an exact node-for-node n8n canvas screenshot. The executable n8n export remains the implementation reference.

5. End-to-end process explained

Step 1 — Generate the test data

Run `python src/generate_data.py`. It generates 1,000 synthetic transactions with 50 injected anomaly labels and saves `data/transactions.csv`.

Step 2 — Validate data quality

Run `python src/data_quality.py`. Checks cover missing columns, transaction IDs, duplicates, invalid amounts, and invalid hours. The latest run passed with zero issues, writing `results/data_quality_report.json`.

Step 3 — Detect and score anomalies

Run `python src/anomaly_detection.py`. IsolationForest uses amount and hour with 100 estimators, contamination 0.05 and random_state 42. Output: `results/anomaly_predictions.csv`.

Step 4 — Serve detection results

Start `uvicorn src.api:app --reload --port 8000`. The route `POST /detect-anomalies` invokes detection and exposes results to the orchestration layer. This is batch processing over a local CSV, not a streaming API.

Step 5 — Trigger and route in n8n

A manual trigger can initiate the workflow; a daily 9 a.m. America/Chicago schedule has been configured but not published. The n8n container accesses host services using `host.docker.internal`.

Step 6 — Store and select records

n8n upserts transaction data into PostgreSQL, selects anomaly candidates via SQL, and uses an alert claim state before email. A Limit node restricts test alert volume.

Step 7 — Generate explanations

The workflow invokes the Ollama HTTP API using the local `llama3.2:3b` model. Generated text provides an explanation for review but is not evidence of fraud; model anomaly scores are not probabilities.

Step 8 — Send and track email

The Gmail branch attempts an alert and records success (`email_sent=true`) or failure (`notification_error`). Claiming reduces repeat attempts but does not guarantee exactly-once email delivery.

6. Two final outputs

Output	What a reviewer can inspect
A. Stored anomaly/alert records	PostgreSQL anomaly_alerts: transaction identifiers, amount, score, AI explanation, claimed/sent state, notification error; plus local CSV/JSON reports.
B. Notification outcome	Gmail alert if delivered; otherwise a logged notification failure. The demonstration has tested sends, but the daily schedule is not activated.

7. Reproduction on a local Mac

The standalone Python pipeline runs independently of n8n. Docker, PostgreSQL, Ollama and Gmail configuration are required only for the full automated demonstration.

```
git clone https://github.com/durgapbattula-sys/ai-financial-anomaly-detection-n8n.git
cd ai-financial-anomaly-detection-n8n
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python src/generate_data.py
python src/data_quality.py
python src/anomaly_detection.py
```

Connect the orchestration components

Start Docker Desktop and the n8n/PostgreSQL containers; start Ollama and pull the specified model; start FastAPI on port 8000. Open n8n at http://localhost:5678 and import workflows/financial-anomaly-final-v1.json. Configure the PostgreSQL credential and Gmail OAuth recipient locally. From Docker, the FastAPI endpoint is http://host.docker.internal:8000/detect-anomalies and Ollama is http://host.docker.internal:11434/api/generate. Test manually before enabling any schedule.

8. Evidence, interpretation and limits

The observed 0.980 precision, recall and F1 are calculated against labels injected into the synthetic dataset and evaluated on that generated batch. They are not estimates of generalization to real transactions. A contamination parameter of 0.05 drives the target anomaly proportion, so 50 flagged observations alone is not an independent performance achievement.

Current limitations: static CSV input; retraining on each request; only amount and hour model features; possible LLM hallucinations; local-only services; no confirmed exactly-once Gmail semantics; schedule intentionally unpublished; no production access control, monitoring, drift evaluation or streaming ingestion.

9. Engineering improvements

Priority	Improvement
High	Add unit/integration tests for generator, quality gates, API response, and database alert claims.
High	Separate training/validation data; evaluate on unseen and more realistic cases.
High	Introduce fresh ingestion and idempotent transaction batch IDs before enabling scheduling.
Medium	Add configuration templates and Docker Compose documentation without secrets.
Medium	Add structured observability and retry/claim expiry policy.
Medium	Ground or template AI explanations and label uncertainty explicitly.

10. Repository and handoff

Source and workflow export: github.com/durgapbattula-sys/ai-financial-anomaly-detection-n8n. Core source files: src/generate_data.py, src/data_quality.py, src/anomaly_detection.py, src/api.py; orchestration export: workflows/financial-anomaly-final-v1.json. Generated data and results remain local/untracked in the documented workflow.